

OPERATING SYSTEMS

Question Bank —Answers

(B.Tech — III Year / V Semester)

UNIT – I: OPERATING SYSTEM OVERVIEW AND SYSTEM STRUCTURES

SECTION A — 2 MARKS

Q1. What is an Operating System?

Ans:

An Operating System (OS) is system software that acts as an intermediary between the computer user and the computer hardware. It manages hardware resources such as CPU, memory, and I/O devices, and provides a convenient, efficient environment in which users can execute their programs.

An OS performs two broad roles:

- **Resource Manager:** Allocates CPU time, memory space, files and I/O devices among competing programs.
- **Control Program:** Controls execution of user programs to prevent errors and improper use of the computer.

Fig 1.1: Operating System as an Interface between User and Hardware

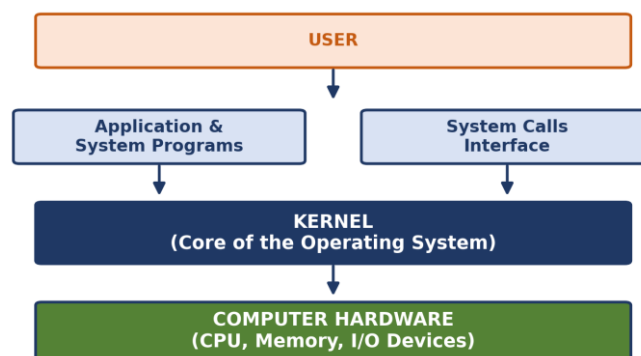


Fig: Operating System as an Interface between User and Hardware

Q2. Define Kernel.

Ans:

Kernel is the core part of the Operating System that is loaded into memory first and stays resident as long as the computer runs. It has direct control over hardware and offers process, memory and device management services to the rest of the system.

Key features:

- It is the first program loaded when the system boots.
- It handles communication between hardware and software.
- Provides essential services: process scheduling, memory management, I/O control.

Q3. What is Multiprogramming?

Ans:

Multiprogramming is a technique where multiple programs reside in memory simultaneously and the CPU switches among them to maximise CPU utilisation.

Key points:

- When one job waits for I/O, the CPU is switched to another job.
- It keeps the CPU busy at all times.
- Increases throughput by reducing idle CPU time.

Q4. How does Dual-Mode Operation work?

Ans:

To protect the OS and hardware from incorrect or malicious user programs, the hardware provides two modes of execution, distinguished by a mode bit:

- **User Mode (mode bit = 1):** Programs execute with restricted privileges; cannot directly access hardware or execute privileged instructions.
- **Kernel / Monitor Mode (mode bit = 0):** The OS executes with full access to hardware and all privileged instructions.

A system call, trap or interrupt switches the mode bit from 1 → 0 (user to kernel). The return-from-interrupt instruction resets it back to 1 (kernel to user).

Fig 1.3: Transition between User Mode and Kernel Mode (Dual-Mode Operation)

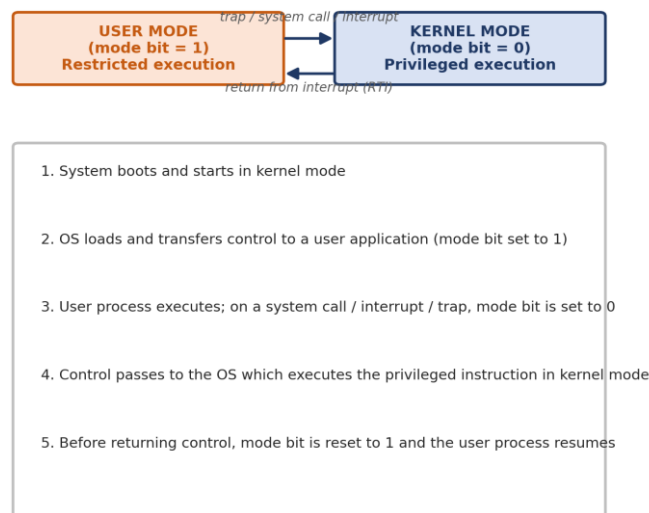


Fig: Transition between User Mode and Kernel Mode

Q5. What are the objectives of an Operating System?

Ans:

The three primary objectives of an Operating System are:

- **Convenience:** Make the computer system easy and pleasant to use.
- **Efficiency:** Manage the resources of the computer system efficiently to maximise throughput.
- **Ability to Evolve:** Permit effective development, testing and introduction of new functions without interfering with existing service.

Q6. What is the use of fork() and exec() system calls?

Ans:

- **fork():** Creates a child process that is a duplicate of the parent process. After fork(), both parent and child execute concurrently.
- **exec():** Replaces a process's memory image with a new program. Typically used after fork() to load a different program into the child process.

Together, `fork()` + `exec()` is the standard method for creating a new process running a different program in UNIX/Linux.

Q7. Define Open Source Operating System.

Ans:

An Open Source Operating System is one whose source code is freely available for use, study, modification, and redistribution by the public.

Characteristics:

- Source code is publicly accessible.
- Can be modified and redistributed by anyone.
- Usually developed collaboratively by a community.

Examples: Linux (Ubuntu, Fedora), FreeBSD, OpenSolaris, Android (based on Linux kernel).

Q8. List the services provided by an Operating System.

Ans:

Services provided to Users:

- User Interface — CLI (Command Line Interface) or GUI (Graphical User Interface).
- Program Execution — Loads a program into memory and runs it.
- I/O Operations — Provides controlled means to perform input/output.
- File-System Manipulation — Create, delete, read, write files/directories.
- Communication — Exchange of information between processes via shared memory or message passing.
- Error Detection — Detects and corrects errors in CPU, memory, I/O devices.

Services for efficient system operation:

- Resource Allocation — Allocates CPU, memory and I/O devices among multiple users.
- Accounting — Tracks resource usage by each user.
- Protection and Security — Ensures controlled access to resources.

Q9. Define System Calls.

Ans:

A System Call is the programming interface through which a running user program requests a service from the operating system's kernel. It is the only legal entry point into the kernel.

Categories of System Calls:

- **Process control:** `fork()`, `exec()`, `exit()`, `wait()`
- **File management:** `open()`, `read()`, `write()`, `close()`
- **Device management:** `ioctl()`, `read()`, `write()`
- **Information maintenance:** `getpid()`, `alarm()`, `sleep()`
- **Communication:** `pipe()`, `shmget()`, `mmap()`
- **Protection:** `chmod()`, `umask()`, `chown()`

Q10. What is the purpose of System Programs?

Ans:

System Programs (System Utilities) provide a convenient environment for program development and execution, allowing users to accomplish common tasks without writing code from scratch.

Categories:

- **File management:** Create, delete, copy, rename, list files.
- **Status information:** Date, time, memory/disk usage.
- **Programming-language support:** Compilers, assemblers, interpreters, debuggers.
- **Program loading & execution:** Loaders, linkers.
- **Communications:** Browsers, e-mail clients, remote login tools.

SECTION B — 10 MARKS

Q1. Explain the different functions of an Operating System and discuss the various services provided by an Operating System.

Ans:

Definition

An Operating System (OS) is system software that acts as an intermediary between the user and the computer hardware. It manages hardware resources (CPU, memory, I/O devices) and provides a convenient environment in which users can execute their programs.

Fig 1.1: Operating System as an Interface between User and Hardware

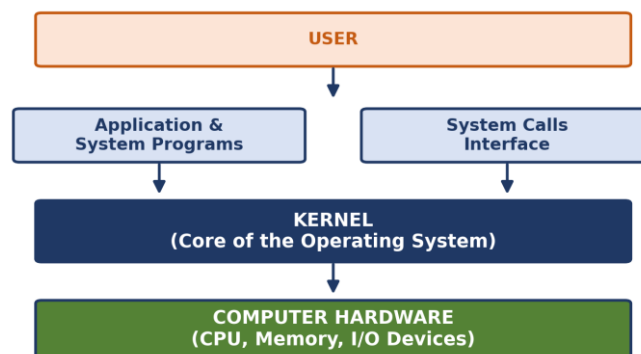


Fig 1.1: OS as an Interface between User and Hardware

Functions of an Operating System

- **Process Management:** Creates, schedules and terminates processes; handles context switching.
- **Memory Management:** Allocates and deallocates memory to processes; handles paging and segmentation.
- **File System Management:** Provides file creation, deletion, reading and writing services.
- **I/O Management:** Controls I/O devices through drivers, buffers and caches.
- **Security & Protection:** Provides authentication and ensures controlled access to resources.
- **Networking:** Manages network protocols and data transfer between systems.

Services Provided to Users

- User Interface — CLI or GUI for interaction.
- Program Execution — loads and runs programs.
- I/O Operations — controlled access to I/O devices.
- File-System Manipulation — manages files and directories.
- Communication — inter-process communication via shared memory or message passing.
- Error Detection — monitors and corrects hardware/software errors.

Services for System Efficiency

- Resource Allocation — allocates CPU, memory and devices fairly among processes.
- Accounting — tracks who uses how much resource.
- Protection & Security — prevents unauthorised access.

Conclusion: The OS therefore forms the foundation layer of every computer system, translating raw hardware capability into a usable, protected, multi-user computing platform.

Q2. Explain Operating System Structures.

Ans:

The internal organisation of an OS can follow several structural approaches, each with different trade-offs between simplicity, performance and reliability.

Fig 1.2: Classification of Operating System Structures

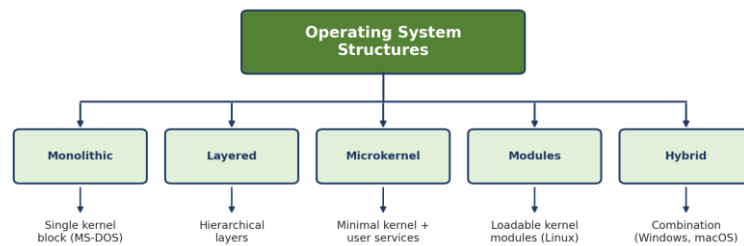


Fig 1.2: Classification of Operating System Structures

1. Simple (Monolithic) Structure

All OS functions are packed into a single large program running in kernel mode, with no clear internal boundaries (e.g., MS-DOS, early UNIX).

- **Advantage:** Fast — no inter-module call overhead.
- **Disadvantage:** Difficult to maintain, extend and debug.

2. Layered Approach

The OS is organised into numbered layers. Layer 0 is the hardware; the topmost layer is the user interface. Each layer can call only the services of the layer directly beneath it.

Fig 1.2: Layered Approach to Operating System Structure

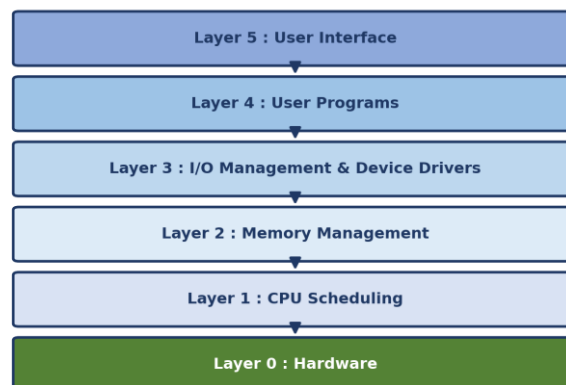


Fig 1.3: Layered Approach to OS Structure

- **Advantage:** Simple to construct and debug each layer independently.
- **Disadvantage:** Layer-to-layer calls add overhead; careful layer definition needed.

3. Microkernel Structure

Removes all non-essential components from the kernel and implements them as user-level programs. The microkernel retains only minimal process and memory management; communication is via message passing.

- **Advantage:** Easier to extend; more reliable and secure since less code runs in kernel mode.
- **Disadvantage:** Extra message-passing overhead reduces performance.

4. Modules (Loadable Kernel Modules)

Uses an object-oriented approach: the kernel has a set of core components and dynamically links additional services through modules at boot time or during run time (e.g., Linux, Solaris).

5. Hybrid Systems

Most modern operating systems (macOS, iOS, Android, Windows) combine two or more of the above approaches to balance performance, modularity and security.

Comparison: Monolithic Kernel vs Microkernel

Basis	Monolithic Kernel	Microkernel
Structure	Single large block of code	Small core + user-space servers
Communication	Direct function calls	Message passing (IPC)
Speed	Faster (no IPC overhead)	Comparatively slower
Reliability	A bug can crash the whole OS	More reliable / fault-isolated
Extensibility	Difficult to extend	Easy to add new services
Examples	Linux, traditional UNIX	QNX, Mach, Minix

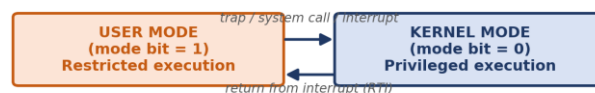
Q3. Explain about the Dual-Mode Operation in OS with a neat block diagram.

Ans:

Concept

To protect the OS and hardware from incorrect or malicious user programs, the hardware provides two modes of execution, distinguished by a mode bit in a special hardware register.

Fig 1.3: Transition between User Mode and Kernel Mode (Dual-Mode Operation)



1. System boots and starts in kernel mode
2. OS loads and transfers control to a user application (mode bit set to 1)
3. User process executes; on a system call / interrupt / trap, mode bit is set to 0
4. Control passes to the OS which executes the privileged instruction in kernel mode
5. Before returning control, mode bit is reset to 1 and the user process resumes

Fig 1.4: Transition between User Mode and Kernel Mode

User Mode (mode bit = 1)

- Programs execute with restricted privileges.
- Cannot directly access hardware or execute privileged instructions.
- Cannot perform I/O directly; must use system calls.

Kernel / Monitor Mode (mode bit = 0)

- The OS executes with full access to all hardware.
- All privileged instructions can be executed.
- Direct hardware access is permitted.

Mode Switching

- User to Kernel: Triggered by system call, trap, or interrupt — mode bit changes from 1 to 0.
- Kernel to User: Return-from-interrupt instruction resets mode bit from 0 to 1.

Importance

Dual-mode operation prevents user code from directly executing privileged instructions such as I/O control and memory-management instructions, thereby protecting the operating system from errant or malicious programs. Without this protection, a user program could take control of the system or corrupt the OS.

Q4. What is Operating System? Explain Computing Environments.

Ans:

Operating System — Definition

An Operating System is system software that acts as an intermediary between the computer user and the computer hardware. It manages resources and provides a convenient environment for program execution.

Computing Environments

An OS must be designed keeping in mind the environment in which it will operate. The major computing environments are:

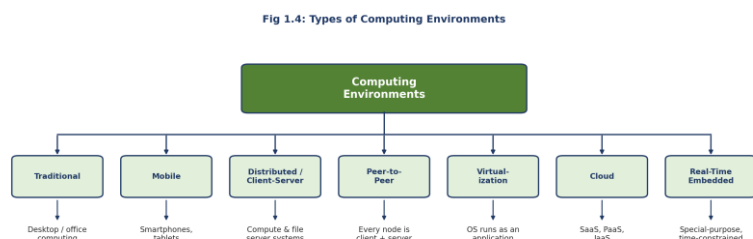


Fig 1.5: Types of Computing Environments

- **Traditional Computing:** Standard office desktops connected via a network running everyday applications such as browsers and word processors.
- **Mobile Computing:** Handheld smartphones and tablets (Android, iOS) offering portable computing with wireless connectivity.
- **Distributed / Client-Server Computing:** Multiple computers connected over a network share resources; servers respond to requests made by clients.
- **Peer-to-Peer (P2P) Computing:** Every node acts as both client and server; nodes join a P2P network to share resources (e.g., BitTorrent).
- **Virtualization:** Abstracts the hardware of one physical computer into several execution environments, allowing an OS to run as an application within another OS (e.g., VMware, VirtualBox).
- **Cloud Computing:** Delivers computing, storage and application services over the Internet, categorised as SaaS, PaaS and IaaS.
- **Real-Time Embedded Systems:** Small, special-purpose computers embedded within devices (car engines, robots), governed by strict real-time constraints.

Q5. Write the differences between Monolithic Kernel and Microkernel.

Ans:

Monolithic Kernel

In a monolithic kernel, the entire OS runs in kernel mode as a single large program. All OS services (process management, memory management, file system, device drivers) are packed into one large block.

- All OS functions share the same address space.
- Communication via direct function calls — very fast.
- Any bug can crash the entire OS.
- Examples: Linux, traditional UNIX.

Microkernel

In a microkernel, only the most essential functions (IPC, basic memory management, basic scheduling) remain in the kernel. All other services run in user space as servers.

- Kernel is small and minimal.

- Communication via message passing (IPC) — adds overhead.
- More reliable and secure — faults are isolated.
- Easier to extend and port to new hardware.
- Examples: QNX, Mach, Minix.

Comparison Table

Basis	Monolithic Kernel	Microkernel
Structure	Single large block of code	Small core + user-space servers
Communication	Direct function calls	Message passing (IPC)
Speed	Faster (no IPC overhead)	Slower due to IPC overhead
Reliability	A bug can crash the whole OS	More reliable / fault-isolated
Extensibility	Difficult to extend	Easy to add new services
Security	Less secure	More secure
Memory usage	More memory usage	Less memory usage
Examples	Linux, traditional UNIX	QNX, Mach, Minix

Q6. Explain briefly System Calls with examples.

Ans:

Definition

A System Call is the programming interface through which a running user program requests a service from the operating system's kernel. It is the only legal entry point into the kernel.

How a System Call Works

1. The user program executes a library call (e.g., `printf()`).
2. The library issues a system call, causing a trap into the kernel.
3. Control transfers to kernel mode; the appropriate kernel routine executes.
4. The result is returned and control passes back to the user program.

Categories of System Calls with Examples

Category	Example Calls
Process control	<code>fork()</code> , <code>exec()</code> , <code>exit()</code> , <code>wait()</code>
File management	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>
Device management	<code>ioctl()</code> , <code>read()</code> , <code>write()</code>
Information maintenance	<code>getpid()</code> , <code>alarm()</code> , <code>sleep()</code>
Communication	<code>pipe()</code> , <code>shmget()</code> , <code>mmap()</code>
Protection	<code>chmod()</code> , <code>umask()</code> , <code>chown()</code>

Q7. Explain briefly User and Operating System Interface.

Ans:

Users interact with the operating system through one of the following interfaces:

1. Command Line Interface (CLI)

- Also called a shell or command interpreter.
- User types text commands directly.
- Fast for experienced users; highly scriptable.
- Low resource usage.

- Examples: bash (Linux), Command Prompt (Windows).

2. Graphical User Interface (GUI)

- Uses windows, icons, menus and a mouse pointer.
- User-friendly and intuitive — suitable for all skill levels.
- More resource-intensive.
- Examples: Windows Desktop, macOS Finder, GNOME (Linux).

3. Touch-Screen Interface

- Used on mobile devices (smartphones, tablets).
- Natural, gesture-based interaction.
- Limited precision compared to keyboard/mouse.
- Examples: Android, iOS.

Comparison Table

Interface	Advantages	Disadvantages
CLI	Fast for experienced users; scriptable; low resource usage	Requires memorising commands; less friendly for beginners
GUI	User-friendly and intuitive	More resource-intensive; less flexible for automation
Touch-Screen	Natural, gesture-based on mobile devices	Limited precision compared to keyboard/mouse

Most modern operating systems provide both a CLI and a GUI, allowing users to choose the interface that best fits the task at hand.

Q8. Discuss about Operating System Services.

Ans:

An operating system provides an environment for execution of programs and services to programs and users. These services can be classified into two groups:

A. Services Provided to Users

- **User Interface:** Provides CLI, GUI or touch-screen interface for user interaction.
- **Program Execution:** Loads a program into memory, runs it and handles normal/abnormal termination.
- **I/O Operations:** Provides controlled means to perform input/output, since user programs cannot access I/O devices directly.
- **File-System Manipulation:** Create, delete, read, write files/directories and manage permissions.
- **Communication:** Processes exchange information via shared memory or message passing.
- **Error Detection:** Detects and corrects errors in the CPU, memory, I/O devices or user programs.

B. Services Ensuring Efficient System Operation

- **Resource Allocation:** Allocates CPU cycles, memory and I/O devices among multiple users or jobs simultaneously.
- **Accounting:** Keeps track of resource usage by each user, useful for billing or usage statistics.
- **Protection and Security:** Ensures controlled access to resources and protects data from unauthorised access; requires authentication.

Q9. Write a short notes on: (a) Operating System Design & Implementation (b) System Programs

Ans:

(a) Operating System Design and Implementation

Designing an OS involves clearly defining two sets of goals:

- **User goals:** Convenient, easy to use, reliable, safe and fast.

- **System goals:** Easy to design, implement and maintain; flexible, reliable, error-free and efficient.

Mechanism vs Policy:

- A mechanism determines how something is done.
- A policy decides what will be done.
- Separating policy from mechanism gives the system flexibility — policy can change without changing the mechanism.

Implementation:

- Early OSes were written in assembly language.
- Modern OSes are written mostly in C/C++, with small time-critical portions in assembly, to allow portability across hardware platforms.

(b) System Programs (System Utilities)

System programs provide a convenient environment for program development and execution, allowing users to accomplish tasks without writing code from scratch.

Category	Examples
File management	Create, delete, copy, rename, list files
Status information	Date, time, memory/disk usage, logging
File modification	Text editors, search/transform tools
Programming-language support	Compilers, assemblers, interpreters, debuggers
Program loading & execution	Loaders, linkers, overlay loaders
Communications	Browsers, e-mail clients, remote login tools
Application programs	Word processors, spreadsheets, database systems

Q10. Discuss about Operating System Debugging.

Ans:

Debugging is the process of finding and fixing errors (bugs) in a system, covering both hardware and software components.

Techniques for OS Debugging

Technique	Purpose
Failure analysis	Error information is written to a log file; the memory state is captured in a core dump for later analysis.
Performance tuning	Treated as a form of debugging aimed at removing system bottlenecks.
Trace listings	Record every system call executed by a process — used with tools such as strace, truss.
Crash dump analysis	A kernel failure produces a crash dump — a capture of kernel memory analysed with a crash dump analyser.
System monitoring tools	Display current activity such as CPU, memory and network usage (e.g., Task Manager, top, vmstat).
DTrace	Dynamically adds probes to a running system (Solaris/UNIX variants) to diagnose performance issues without recompiling.

Performance tuning involves removing performance bottlenecks, often identified using profiling tools that record where in the code the program spends its time. The ultimate goal of debugging is to find and correct errors to improve system reliability and performance.

UNIT – II: PROCESS, THREADS & CONCURRENCY, CPU SCHEDULING

SECTION A — 2 MARKS

Q1. Define Process.

Ans:

A Process is a program in execution. It is an active entity comprising:

- Program code (text section)
- Current activity (program counter, registers)
- Stack (temporary data — function parameters, local variables)
- Data section (global variables)
- Heap (dynamically allocated memory)

A program is a passive entity stored on disk; a process is the active execution of that program.

Q2. What is meant by the State of a Process?

Ans:

The state of a process describes its current activity. A process may be in one of the following states:

- **New:** The process is being created.
- **Ready:** The process is waiting to be assigned to the CPU.
- **Running:** Instructions are being executed by the CPU.
- **Waiting:** The process is waiting for some event (e.g., I/O completion).
- **Terminated:** The process has finished execution.

Fig 2.1: Process State Transition Diagram

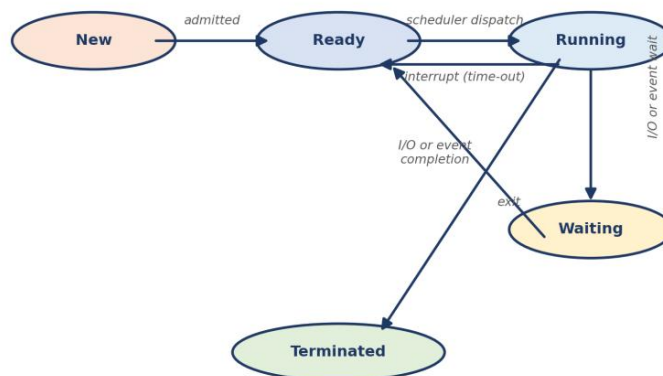


Fig 2.1: Process State Transition Diagram

Q3. Define Schedulers.

Ans:

Schedulers are OS modules that select processes for execution or loading into memory. There are three main types:

- **Long-Term Scheduler (Job Scheduler):** Selects processes from the job pool on disk and loads them into memory; controls the degree of multiprogramming.
- **Short-Term Scheduler (CPU Scheduler):** Selects among processes in the ready queue and allocates the CPU; executes very frequently.
- **Medium-Term Scheduler:** Swaps processes out of memory to reduce the degree of multiprogramming and swaps them back in later.

Q4. What requirements must be satisfied for a solution to the Critical Section Problem?

Ans:

Any valid solution to the critical section problem must satisfy three requirements:

- **Mutual Exclusion:** If a process is executing in its critical section, no other process may execute in its critical section.
- **Progress:** If no process is in its critical section and some processes wish to enter, the decision on who enters next cannot be postponed indefinitely.
- **Bounded Waiting:** There must be a limit on how many times other processes may enter their critical section after a process has requested entry and before that request is granted.

Q5. What are the types of Scheduler?

Ans:

There are three types of schedulers:

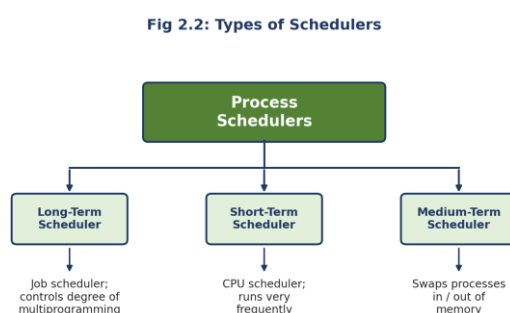


Fig 2.2: Types of Schedulers

- **Long-Term Scheduler (Job Scheduler):** Controls degree of multiprogramming; executes infrequently (seconds/minutes).
- **Short-Term Scheduler (CPU Scheduler):** Allocates CPU from ready queue; executes very frequently (milliseconds).
- **Medium-Term Scheduler:** Performs swapping — moves processes between memory and disk to balance load.

Q6. Define Thread.

Ans:

A Thread is the basic unit of CPU utilisation, also called a Lightweight Process (LWP). It comprises:

- Its own thread ID, program counter, register set and stack.
- Shares the code section, data section and OS resources (e.g., open files) with other threads of the same process.

Threads allow multiple tasks within a single process to execute concurrently, improving performance and responsiveness.

Q7. What does PCB contain?

Ans:

The Process Control Block (PCB) is a data structure maintained by the OS for each process. It contains:

- Process state (New, Ready, Running, Waiting, Terminated)
- Process number / PID (Process Identifier)
- Program counter (address of next instruction to execute)
- CPU registers (contents of all process-centric registers)
- CPU scheduling information (priority, scheduling queue pointers)

- Memory-management information (page tables, segment tables)
- Accounting information (CPU used, clock time elapsed)
- I/O status information (list of I/O devices allocated, open files)

Q8. What is a Multiprocessor system?

Ans:

A Multiprocessor system is a system with two or more CPUs (processors) sharing common memory, buses, clock and peripheral devices, working simultaneously to increase throughput and reliability.

Types:

- **Symmetric Multiprocessing (SMP):** Each processor runs an identical copy of the OS and handles its own scheduling.
- **Asymmetric Multiprocessing:** One master processor controls the system and assigns work to slave processors.

Advantages: Increased throughput, economy of scale, increased reliability.

Q9. Define Context Switching.

Ans:

Context Switching is the act of saving the state (context) of the currently running process in its PCB and loading the saved state of the next process to be run, so that the CPU can switch between processes.

Key points:

- The context of a process is represented in its PCB.
- Context-switch time is pure overhead — no useful work is done during switching.
- Its speed varies from machine to machine depending on memory speed, number of registers and special instructions.

Q10. List the Thread Libraries.

Ans:

Thread libraries provide programmers with an API for creating and managing threads. The three main thread libraries are:

- **POSIX Pthreads:** A standard API for thread creation and synchronization used in UNIX/Linux systems.
- **Win32 Threads:** The thread library available in the Windows API for Windows operating systems.
- **Java Threads:** The Java programming language and Java Virtual Machine (JVM) provide thread creation and management using the Thread class and Runnable interface.

SECTION B — 10 MARKS

Q1. (a) Define Process. Explain Process State Diagram. (b) Explain about Process Schedulers.

Ans:

(a) Process and Process State Diagram

A Process is a program in execution — an active entity comprising program code, current activity (PC, registers), stack, data section and heap.

Process States

A process passes through the following five states during its lifetime:

- **New:** The process is being created.
- **Ready:** The process is in memory, waiting to be assigned to the CPU.
- **Running:** CPU is executing the process instructions.
- **Waiting:** The process is waiting for some event such as I/O completion or a signal.

- **Terminated:** The process has completed execution.

Fig 2.1: Process State Transition Diagram

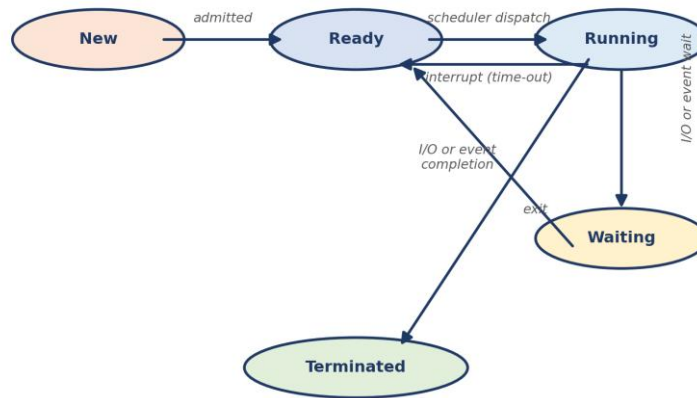


Fig 2.1: Process State Transition Diagram

State Transitions

- New → Ready: Process is admitted into memory.
- Ready → Running: CPU scheduler dispatches the process.
- Running → Waiting: Process requests I/O or waits for an event.
- Waiting → Ready: I/O completes; process moves back to ready queue.
- Running → Ready: Interrupt occurs (preemptive scheduling).
- Running → Terminated: Process finishes execution.

Process Control Block (PCB)

Each process is represented in the OS by a PCB which stores: process state, PID, program counter, CPU registers, scheduling info, memory-management info, accounting info, and I/O status.

(b) Process Schedulers

Schedulers are OS modules that select processes for execution or loading.

Fig 2.2: Types of Schedulers

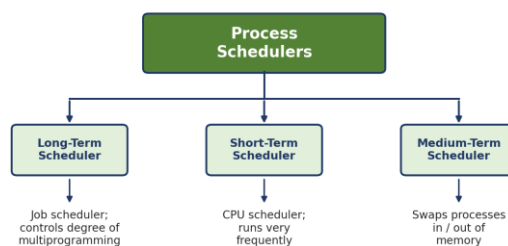


Fig 2.2: Types of Schedulers

- **Long-Term Scheduler (Job Scheduler):** Selects processes from the job pool on disk and loads them into main memory. Controls the degree of multiprogramming. Executes infrequently (every few seconds or minutes).
- **Short-Term Scheduler (CPU Scheduler):** Selects among the processes in the ready queue and allocates the CPU to one. Must be very fast since it executes frequently (every ~100 ms).
- **Medium-Term Scheduler:** Performs swapping — removes processes from memory (swap out) to reduce multiprogramming degree; brings them back (swap in) when conditions allow.

Context Switching

When the CPU switches from one process to another, the OS saves the state of the old process in its PCB and loads the saved state of the new process. Context-switch time is overhead — no useful work is done during it.

Q2. Consider 3 processes P1, P2 and P3 requiring 5, 7, 4 time units arriving at 0, 1, 3. Draw Gantt chart and find average waiting time for: (i) Round Robin (quantum=2) (ii) FCFS.

Ans:

Process Information

Process	Burst Time	Arrival Time
P1	5	0
P2	7	1
P3	4	3

(i) Round Robin (Quantum = 2)

Gantt Chart execution order:

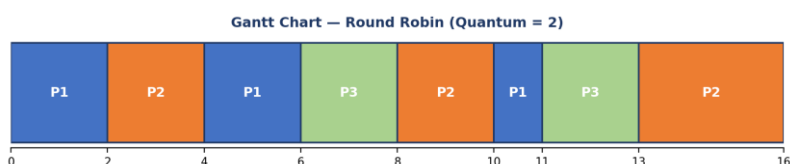


Fig: Round Robin Gantt Chart (q=2) for P1, P2, P3

Execution sequence: P1(0-2) → P2(2-4) → P1(4-6) → P3(6-8) → P2(8-10) → P3(10-12) → P1(12-13) → P2(13-16)

Process	Burst	Arrival	Completion	Turnaround	Waiting
P1	5	0	11	11	6
P2	7	1	16	15	8
P3	4	3	13	10	6

Average Waiting Time (RR, q=2) = (6 + 8 + 6) / 3 = 6.67 time units

(ii) FCFS (First-Come, First-Served)

Execution order by arrival: P1 → P2 → P3

Process	Burst	Arrival	Start	Completion	Turnaround	Waiting
P1	5	0	0	5	5	0
P2	7	1	5	12	11	4
P3	4	3	12	16	13	9

Average Waiting Time (FCFS) = (0 + 4 + 9) / 3 = 4.33 time units

Q3. Explain CPU Scheduling Algorithms with examples.

Ans:

CPU Scheduling determines which process in the ready queue should be allocated to the CPU. The major algorithms are:

Fig 2.3: Classification of CPU Scheduling Algorithms

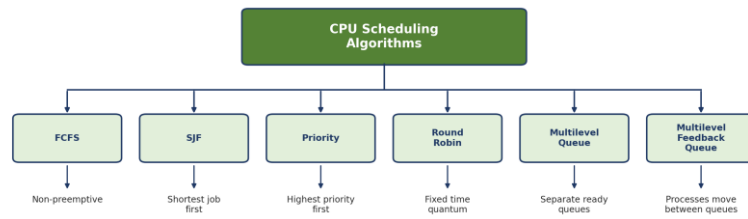


Fig 2.3: Classification of CPU Scheduling Algorithms

1. First-Come, First-Served (FCFS)

- Non-preemptive; processes are executed in arrival order.
- Simple to implement using a FIFO queue.
- Disadvantage: Convoy effect — short processes wait behind a long one.

Example: P1(24), P2(3), P3(5) all arrive at t=0:

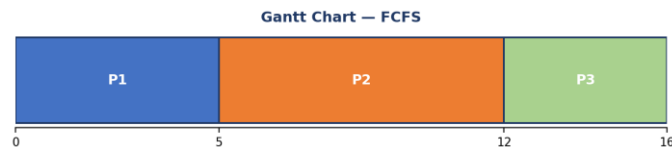


Fig: FCFS Gantt Chart

$$\text{Average Waiting Time} = (0 + 24 + 27) / 3 = 17 \text{ ms}$$

2. Shortest Job First (SJF)

- Selects the process with the smallest burst time next.
- Can be preemptive (SRTF) or non-preemptive.
- Gives minimum average waiting time, but requires knowledge of burst time.

Example: P1(8,0), P2(4,1), P3(9,2), P4(5,3):

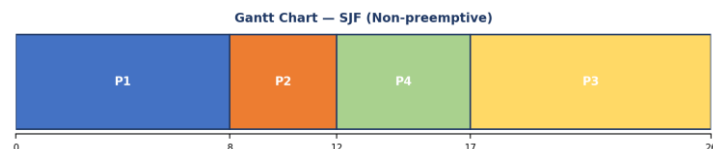


Fig: SJF Gantt Chart

$$\text{Average Waiting Time (SJF)} = (0 + 7 + 9 + 15) / 4 = 7.75 \text{ ms}$$

3. Priority Scheduling

- The CPU is allocated to the process with the highest priority.
- Can be preemptive or non-preemptive.
- Problem: Starvation of low-priority processes; Solution: Ageing.

4. Round Robin (RR)

- Each process gets a fixed time quantum; preempted and placed at the back of the queue when quantum expires.
- Ideal for time-sharing systems; provides fair CPU distribution.
- Performance depends on the size of the time quantum.

Example (q=2): P1(5,0), P2(7,1), P3(4,3):

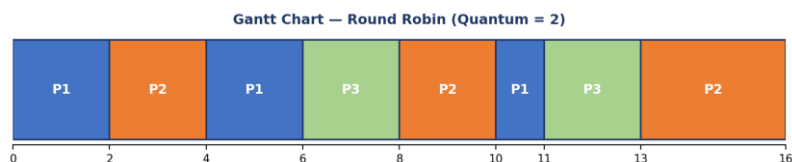


Fig: Round Robin Gantt Chart (q=2)

Average Waiting Time (RR, q=2) = $(6 + 8 + 6) / 3 = 6.67$ ms

5. Multilevel Queue Scheduling

- Ready queue is divided into separate queues (e.g., foreground and background).
- Each queue has its own scheduling algorithm.
- Processes cannot move between queues.

6. Multilevel Feedback Queue Scheduling

- Similar to multilevel queue, but processes may move between queues based on their CPU-burst behaviour.
- Most flexible and complex scheduling algorithm.

Q4. (a) Explain about Scheduling Criteria. (b) Evaluate FCFS CPU Scheduling for P1(24), P2(3), P3(5), P4(6) all arriving at t=0.

Ans:

(a) Scheduling Criteria

The following criteria are used to compare and evaluate CPU scheduling algorithms:

- **CPU Utilisation:** Keep the CPU as busy as possible; ideally 40%–90%.
- **Throughput:** Number of processes completed per unit time. Maximise this.
- **Turnaround Time:** Total time from submission to completion. Minimise this. Formula: Turnaround = Completion Time – Arrival Time.
- **Waiting Time:** Total time a process spends in the ready queue. Minimise this. Formula: Waiting Time = Turnaround Time – Burst Time.
- **Response Time:** Time from submission until the first response is produced. Minimise this.

(b) FCFS Evaluation

Processes: P1 (Burst=24), P2 (Burst=3), P3 (Burst=5), P4 (Burst=6) — all arrive at t=0.

Execution order: P1 → P2 → P3 → P4

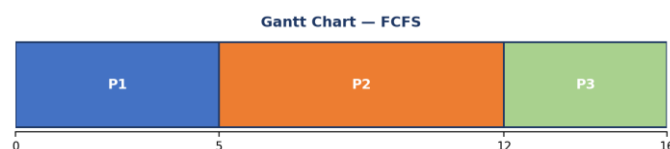


Fig: FCFS Gantt Chart for P1, P2, P3, P4

Process	Burst	Start	Completion	Turnaround	Waiting
P1	24	0	24	24	0
P2	3	24	27	27	24
P3	5	27	32	32	27
P4	6	32	38	38	32

Average Waiting Time (FCFS) = $(0 + 24 + 27 + 32) / 4 = 20.75$ time units

Observation: The convoy effect is clearly seen — P2, P3 and P4 all wait a long time behind the long process P1.

Q5. Evaluate SJF CPU Scheduling for P1(8,0), P2(4,1), P3(9,2), P4(5,3).

Ans:

Algorithm: Non-preemptive SJF — at each decision point, pick the process with the shortest burst time from those that have arrived.

Process	Burst Time	Arrival Time
P1	8	0

P2	4	1
P3	9	2
P4	5	3

Execution order: P1(0-8) → P2(8-12) → P4(12-17) → P3(17-26)

At t=0: only P1 has arrived → run P1. At t=8: P2(4), P3(9), P4(5) available → pick P2. At t=12: P3(9), P4(5) → pick P4. At t=17: only P3 → run P3.

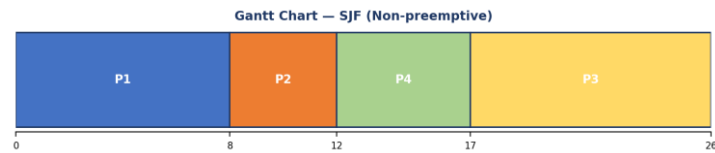


Fig: SJF Gantt Chart

Process	Burst	Arrival	Completion	Turnaround	Waiting
P1	8	0	8	8	0
P2	4	1	12	11	7
P4	5	3	17	14	9
P3	9	2	26	24	15

Average Waiting Time (SJF) = $(0 + 7 + 9 + 15) / 4 = 7.75$ time units

Note: SJF gives the minimum average waiting time among all non-preemptive algorithms.

Q6. Evaluate Round Robin CPU Scheduling (Time Slice = 3 ms) for P1(10,5), P2(5,3), P3(18,0), P4(6,4).

Ans:

Process	Burst Time	Arrival Time
P1	10	5
P2	5	3
P3	18	0
P4	6	4

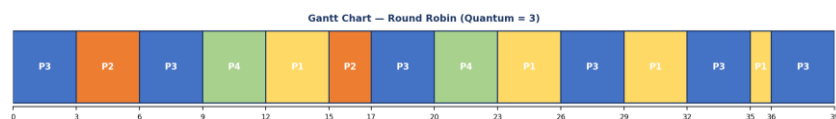


Fig: Round Robin Gantt Chart ($q=3$) for P1, P2, P3, P4

Execution order: P3(0-3) → P3(3-6) [P2,P4 arrive] → P2(6-9) → P4(9-12) → P3(12-15) [P1 arrives] → P1(15-18) → P3(18-21) → P2(21-23) → P4(23-26) → P3(26-29) → P1(29-32) → P3(32-35) → P4(35-37 done) → P1(37-39) → P3(39-42 done)...

Process	Burst	Arrival	Completion	Turnaround	Waiting
P2	5	3	17	14	9
P4	6	4	23	19	13
P1	10	5	36	31	21
P3	18	0	39	39	21

Average Waiting Time (RR, $q=3$) = $(9 + 13 + 21 + 21) / 4 = 16$ time units

Q7. Explain in detail Inter-Process Communication (IPC).

Ans:

Inter-Process Communication (IPC) is a mechanism that allows cooperating processes to exchange data and information. Two fundamental models are used:

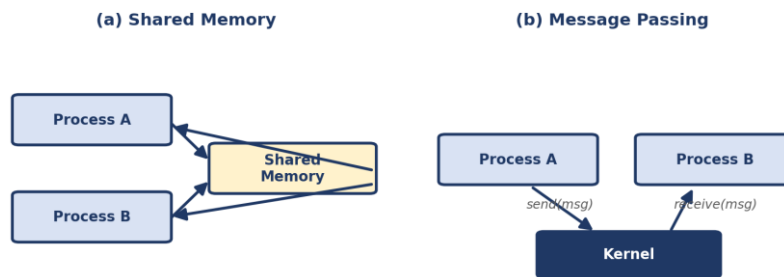


Fig 2.4: Shared Memory vs Message Passing

1. Shared Memory Systems

A region of memory is shared between cooperating processes. They exchange information by reading and writing to this shared region.

- **Advantage:** Fast — no kernel intervention needed for each data exchange after the shared memory region is established.
- **Disadvantage:** Requires explicit synchronisation (e.g., using semaphores) to avoid race conditions.
- **Example:** Producer-Consumer problem — producer places items in buffer; consumer removes items.

2. Message Passing Systems

Processes communicate by exchanging messages via the OS using `send(message)` and `receive(message)`, without sharing an address space.

- **Advantage:** Useful in distributed systems; no synchronisation problem.
- **Disadvantage:** Slower — every message exchange requires kernel involvement.

Message Passing can be:

- Direct or Indirect communication.
- Synchronous (blocking) or Asynchronous (non-blocking).
- Zero, bounded or unbounded buffering.

Comparison

Basis	Shared Memory	Message Passing
Speed	Fast (no kernel per exchange)	Slower (kernel involvement)
Synchronization	Must be handled by programmer	Handled by OS
Useful for	Large data on same machine	Distributed systems
Example	Pipes, shared buffers	Sockets, MPI

Q8. Discuss about Threading Issues.

Ans:

When using threads, several important issues must be considered:

1. `fork()` and `exec()` System Calls

When a multi-threaded process calls `fork()`, there is a question: should `fork()` duplicate only the calling thread, or duplicate every thread in the process? Some UNIX systems provide two versions of `fork()` to handle both cases.

2. Signal Handling

Signals (e.g., SIGINT, SIGSEGV) may be delivered in different ways in multi-threaded programs:

- To the thread to which the signal applies.
- To every thread in the process.
- To selected threads in the process.
- To one specific thread designated to receive all signals.

3. Thread Cancellation

Terminating a thread before it finishes. Two approaches:

- **Asynchronous cancellation:** Immediately terminates the target thread.
- **Deferred cancellation:** The target thread periodically checks whether it should terminate at cancellation points.

4. Thread-Local Storage (TLS)

Allows each thread to keep its own private copy of data not shared with other threads of the same process. Useful when a global variable must have different values in each thread.

5. Scheduler Activations

A communication mechanism between the user-thread library and the kernel to maintain an appropriate number of kernel threads, using an intermediate data structure — the Lightweight Process (LWP).

Q9. Write about Operations on Processes.

Ans:

Process Creation

A parent process can create new child processes using the fork() system call, forming a tree of processes.

- The parent and child may share all resources, a subset, or none.
- The parent and child may execute concurrently, or the parent may wait for the child to terminate.
- exec() is typically used after fork() to replace the child's memory image with a new program.

Resource sharing options:

- Parent and child share all resources.
- Child shares a subset of parent's resources.
- Parent and child share no resources.

Process Termination

A process terminates when it finishes execution and calls exit(), returning its exit status to the parent via wait().

A parent may also abort a child process if:

- The child exceeds its allocated resources.
- The assigned task is no longer required.
- The parent itself is exiting and cascading termination is not permitted.

Special Cases

- **Orphan Process:** A process whose parent has terminated while the process itself is still running. The OS (e.g., init process in Linux) adopts orphan processes.
- **Zombie Process:** A process that has terminated but whose exit status has not yet been collected by the parent via wait(). It occupies a slot in the process table.

Q10. What is a Thread? Discuss about Multithreading Models.

Ans:

Thread — Definition

A Thread is the basic unit of CPU utilisation (also called a Lightweight Process). It has its own thread ID, program counter, register set and stack, while sharing the code section, data section and OS resources with other threads of the same process.

Benefits of Multithreading

- Responsiveness — allows a program to continue running even if part of it is blocked.
- Resource Sharing — threads share memory and resources of the process by default.
- Economy — creating a thread is cheaper than creating a process.
- Scalability — threads can run in parallel on multiprocessor systems.

Multithreading Models

These models describe the relationship between user threads and kernel threads:

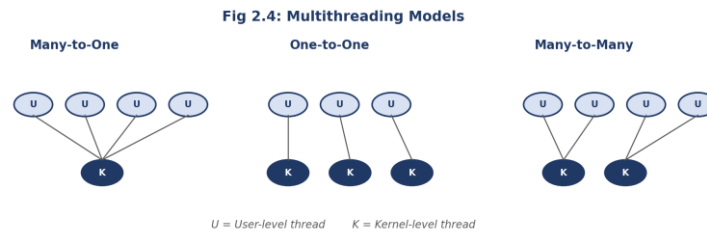


Fig 2.5: Multithreading Models

- **Many-to-One:** Many user threads map to one kernel thread. If one user thread makes a blocking system call, the entire process blocks. Cannot exploit multiple cores. Example: Green threads.
- **One-to-One:** Each user thread maps to its own kernel thread. Provides more concurrency and true parallelism on multiprocessors, but creation overhead is high. Examples: Windows, Linux.
- **Many-to-Many:** Multiplexes many user threads onto a smaller or equal number of kernel threads. Combines flexibility with real parallelism. Example: Windows with ThreadFiber.

UNIT – III: PROCESS SYNCHRONIZATION & DEADLOCKS

SECTION A — 2 MARKS

Q1. Define Deadlock.

Ans:

A Deadlock is a situation in which two or more processes are blocked forever, each waiting for a resource held by another process in the same set, so that none of them can proceed.

Conditions for Deadlock (Coffman Conditions — all four must hold simultaneously):

- **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
- **Hold and Wait:** A process holds at least one resource and is waiting for additional resources held by other processes.
- **No Preemption:** Resources cannot be forcibly taken from a process; they must be voluntarily released.
- **Circular Wait:** A set of processes $\{P_0, P_1, \dots, P_n\}$ exists such that P_0 waits for P_1 , P_1 waits for P_2 , ..., P_n waits for P_0 .

Q2. Define Race Condition.

Ans:

A Race Condition is a situation where multiple processes or threads access and manipulate shared data concurrently, and the final result depends on the particular order in which the accesses take place, leading to inconsistent or incorrect results.

Example: Two processes P_1 and P_2 both increment a shared variable count.

- If both read count (say 5) at the same time, increment it locally to 6, and write it back,
- The final value will be 6 instead of the correct value 7.

Solution: Use synchronisation mechanisms such as semaphores, mutex locks or monitors to ensure only one process accesses the shared data at a time.

Q3. What are the requirements that a solution to the Critical Section problem must satisfy?

Ans:

Any valid solution to the critical section problem must satisfy three requirements:

- **1. Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections. Only one process at a time.
- **2. Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, the selection of the next process to enter cannot be postponed indefinitely. Processes must make progress.
- **3. Bounded Waiting:** There must be a bound (limit) on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted. No process should wait forever.

Q4. Define Semaphores.

Ans:

A Semaphore is an integer synchronisation variable S that, apart from initialisation, is accessed only through two atomic (indivisible) operations: `wait()` and `signal()`.

Operations:

```
wait(S) {                               signal(S) {
    while (S <= 0)                        S++;
    ; // busy wait }
    S--;
```

Types of Semaphores:

- **Counting Semaphore:** Value may range over an unrestricted domain. Used to control access to a resource with multiple instances (e.g., controlling access to a pool of database connections).
- **Binary Semaphore (Mutex Lock):** Value is restricted to 0 or 1. Used to implement mutual exclusion — equivalent to a mutex lock.

Q5. Name the Classic Problems of Synchronization.

Ans:

The three classic problems of synchronization are:

Fig 3.2: Classic Problems of Process Synchronization

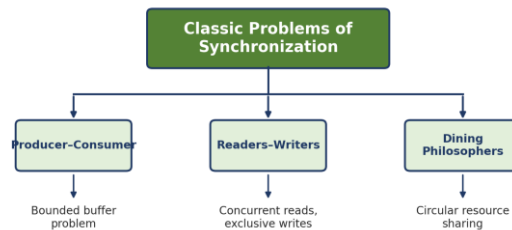


Fig 3.1: Classic Problems of Process Synchronization

- **1. Producer–Consumer (Bounded Buffer) Problem:** A producer places items in a shared buffer and a consumer removes items. The producer must wait if the buffer is full; the consumer must wait if empty.
- **2. Readers–Writers Problem:** Multiple readers can read shared data concurrently, but a writer requires exclusive access. The challenge is to avoid starvation of either readers or writers.
- **3. Dining Philosophers Problem:** Five philosophers alternately think and eat, sharing five chopsticks. The challenge is to avoid deadlock and starvation while ensuring each philosopher can eat.

Q6. What is the Critical Section Problem?

Ans:

A Critical Section is a segment of code in which a process may be changing common variables, updating a table, writing a file, or accessing shared resources. No two processes may execute in their critical sections at the same time.

The Critical Section Problem is the challenge of designing a protocol that lets processes cooperate safely while accessing shared resources.

General structure of a process:

```

do {
    entry section      // request permission to enter
    critical section   // shared resource accessed here
    exit section       // release the critical section
    remainder section  // non-critical code
} while (true);
  
```

The solution must satisfy Mutual Exclusion, Progress, and Bounded Waiting.

Q7. Define Busy Waiting and Spinlock.

Ans:

- **Busy Waiting:** Also called spinning — a process continuously checks (loops) a condition to see if it can enter the critical section, without relinquishing the CPU. It wastes CPU cycles but avoids the overhead of a context switch.
- **Spinlock:** A semaphore or mutex implemented using busy waiting. When a process tries to acquire a lock that is already held by another, it spins in a loop waiting for the lock to be released. Spinlocks are useful when the expected wait time is short (less than the context-switch overhead).

Disadvantage of Busy Waiting: Wastes CPU cycles that could be used productively by other processes.

Solution: Block the process (put it to sleep) and wake it up when the condition is satisfied — this is implemented in blocking semaphores.

Q8. What is Process Synchronization?

Ans:

Process Synchronization is the set of mechanisms used to coordinate the execution of cooperating processes that share data, so as to preserve data consistency and avoid race conditions.

Why it is needed:

- When multiple processes share data and execute concurrently, the outcome can depend on the order of execution.
- Without synchronisation, data inconsistency and race conditions can occur.

Synchronisation Mechanisms:

- **Semaphores:** Integer variables accessed through wait() and signal() operations.
- **Mutex Locks:** Binary locks ensuring mutual exclusion.
- **Monitors:** High-level constructs that encapsulate shared data with procedures.
- **Peterson's Solution:** Software-based solution for two processes.

SECTION B — 10 MARKS

Q1. What is Critical Section Problem? Explain with example.

Ans:

Definition

A Critical Section is a segment of code in which a process accesses shared resources (common variables, tables, files). No two processes may execute in their critical sections simultaneously.

The Critical Section Problem is the challenge of designing an entry and exit protocol that guarantees safe access to shared resources.

General Structure

```
do {  
    entry section      // request to enter critical section  
    critical section   // only one process here at a time  
    exit section       // release the critical section  
    remainder section  // non-critical code  
} while (true);
```

Requirements for a Valid Solution

- **Mutual Exclusion:** If P_i is in its critical section, no other P_j may be in its critical section.
- **Progress:** If no process is in the critical section and some want to enter, selection cannot be postponed indefinitely.
- **Bounded Waiting:** A limit must exist on how many times others enter the CS after a process requests entry.

Example — Race Condition

Consider two processes P1 and P2 both executing: $\text{count} = \text{count} + 1$ (count starts at 5):

```
P1: register1 = count      // register1 = 5  
P2: register2 = count      // register2 = 5  
P1: register1 = register1+1 // register1 = 6  
P2: register2 = register2+1 // register2 = 6  
P1: count = register1      // count = 6  
P2: count = register2      // count = 6 (WRONG! Should be 7)
```

This demonstrates a race condition where the final value of count is incorrect because both processes read the old value before writing the updated value. The critical section mechanism prevents this.

Solutions

- Software: Peterson's Solution (for 2 processes).
- Hardware: TestAndSet(), CompareAndSwap() atomic instructions.
- OS-level: Semaphores, Mutex locks, Monitors.

Q2. What is Semaphore? Explain Producer-Consumer problem using Semaphore.

Ans:

Semaphore — Definition

A Semaphore is an integer synchronisation variable S, accessed only through two indivisible (atomic) operations:

```
wait(S) {                               signal(S) {
    while (S <= 0)                        S++;
    ; // spin                            }
    S--;
}
```

- Counting Semaphore: Value ranges over an unrestricted domain; used to control access to a pool of resources.
- Binary Semaphore (Mutex): Value is 0 or 1; implements mutual exclusion.

Producer-Consumer (Bounded Buffer) Problem

A producer places items into a shared buffer of size n. A consumer removes items. The problem is to ensure the producer does not produce when the buffer is full, and the consumer does not consume when the buffer is empty.

Three semaphores are used:

- **mutex**: Initialised to 1 — ensures mutual exclusion when accessing the buffer.
- **empty**: Initialised to n (buffer size) — counts empty slots; producer waits if 0.
- **full**: Initialised to 0 — counts filled slots; consumer waits if 0.

Solution Code

Fig 3.2: Classic Problems of Process Synchronization

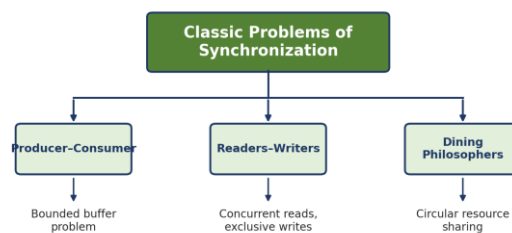


Fig: Classic Synchronization Problems

```
PRODUCER:
do {
    produce an item;
    wait(empty);
    wait(mutex);
    add item to buffer;
    signal(mutex);
    signal(full);
} while (true);
```

```
CONSUMER:
do {
    wait(full);
    wait(mutex);
    remove item from buffer;
    signal(mutex);
    signal(empty);
    consume the item;
} while (true);
```

Explanation:

- Producer calls wait(empty) to check if there is an empty slot; if buffer full, producer blocks.
- Producer calls wait(mutex) to get exclusive access to the buffer.
- After adding the item, producer calls signal(mutex) to release buffer and signal(full) to indicate one more full slot.

- Consumer calls wait(full) to check if there is a filled slot; if buffer empty, consumer blocks.
- Consumer calls wait(mutex) to get exclusive access, removes an item, releases mutex, and signals empty.

This ensures: no buffer overflow or underflow, mutual exclusion in buffer access, and correct synchronisation between producer and consumer.

Q3. Define Process Synchronization and explain Peterson's Solution.

Ans:

Process Synchronization

Process Synchronization is the set of mechanisms used to coordinate the execution of cooperating processes sharing data, to preserve data consistency and prevent race conditions.

Peterson's Solution

Peterson's Solution is a classic software-based solution to the critical-section problem for two processes P_i and P_j . It uses two shared variables:

- **turn:** An integer whose value indicates whose turn it is to enter the critical section.
- **flag[2]:** A boolean array where $\text{flag}[i] = \text{true}$ means process P_i is ready to enter its critical section.

Algorithm for Process P_i

```
do {
    flag[i] = true;           //  $P_i$  is ready to enter
    turn = j;                // give the turn to  $P_j$ 
    while (flag[j] && turn == j)
        ; // busy wait - if  $P_j$  is ready AND it is  $P_j$ 's turn,  $P_i$  waits

    // *** critical section ***

    flag[i] = false;         //  $P_i$  is done; exit section

    // remainder section
} while (true);
```

Proof of Correctness

- **Mutual Exclusion:** P_i enters its CS only if $\text{flag}[j]$ is false OR $\text{turn} = i$. Both conditions cannot be violated simultaneously for both processes.
- **Progress:** If P_j is not ready ($\text{flag}[j] = \text{false}$), P_i enters immediately. If both are ready, turn decides.
- **Bounded Waiting:** P_i waits at most one entry by P_j before P_i gets the turn.

Limitations

- Works only for two processes.
- On modern architectures with instruction reordering, memory barriers may be required.
- Uses busy waiting (spinlock) which wastes CPU.

Q4. What is Monitor? Explain with any example using Monitor.

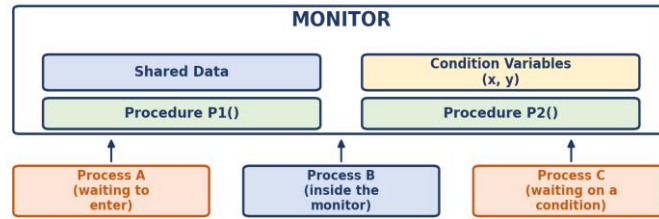
Ans:

Monitor — Definition

A Monitor is a high-level synchronisation construct — an abstract data type that encapsulates shared data together with the procedures that operate on it, guaranteeing that only one process can be active inside the monitor at any instant.

The key advantage over semaphores is that mutual exclusion is automatically enforced by the monitor — the programmer does not need to call wait() and signal() explicitly for mutual exclusion.

Fig 3.5: Structure of a Monitor



Only ONE process may be active inside the monitor at any instant.
Others wait in the entry queue or on a condition variable.

Fig 3.3: Structure of a Monitor

Condition Variables

A monitor uses condition variables (e.g., x, y) with two operations:

- **x.wait():** Suspends the calling process until another process calls x.signal(). The suspended process is placed in a queue associated with condition x.
- **x.signal():** Resumes exactly one suspended process that is waiting on condition x. If no process is waiting, signal() has no effect.

Monitor Syntax

```

monitor MonitorName {
    // shared variable declarations
    condition x, y;

    procedure P1(...) { ... }
    procedure P2(...) { ... }

    initialization code(...) { ... }
}

```

Example: Dining Philosophers Using a Monitor

A monitor DiningPhilosophers maintains the state of each philosopher using an enum: THINKING, HUNGRY, or EATING.

```

monitor DiningPhilosophers {
    enum { THINKING, HUNGRY, EATING } state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);          // try to eat
        if (state[i] != EATING)
            self[i].wait(); // wait if cannot eat
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i+4) % 5); // check left neighbor
        test((i+1) % 5); // check right neighbor
    }

    void test(int i) {
        if (state[(i+4)%5] != EATING &&
            state[i] == HUNGRY &&
            state[(i+1)%5] != EATING) {
            state[i] = EATING;
        }
    }
}

```

```

        self[i].signal(); // wake up philosopher i
    }
}
}

```

Each philosopher calls `pickup(i)` before eating and `putdown(i)` after eating. A philosopher can eat only when neither neighbour is eating, preventing deadlock and starvation.

Q5. Explain the solution for the Dining-Philosophers Problem.

Ans:

Problem Statement

Five philosophers sit around a circular table with five chopsticks (one between each pair of neighbours). Each philosopher alternately thinks and eats. To eat, a philosopher needs both the chopstick on the left and the one on the right.

Fig 3.6: Dining Philosophers Problem (5 Philosophers, 5 Chopsticks)

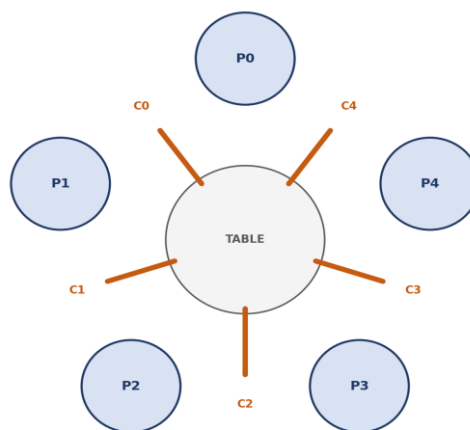


Fig 3.2: The Dining Philosophers Problem

Challenges

- **Deadlock:** All philosophers pick up their left chopstick simultaneously — no one can pick up the right chopstick.
- **Starvation:** A philosopher may wait indefinitely if neighbours keep eating alternately.

Solution 1 — Semaphore-Based Solution

Represent each chopstick with a semaphore. A philosopher picks up a chopstick by executing `wait()` and puts it down by executing `signal()`.

```

semaphore chopstick[5]; // all initialised to 1

Philosopher i:
do {
    // THINK
    wait(chopstick[i]);           // pick up left
    wait(chopstick[(i+1) % 5]); // pick up right
    // EAT
    signal(chopstick[i]);        // put down left
    signal(chopstick[(i+1) % 5]); // put down right
} while (true);

```

Problem: This can cause deadlock if all 5 philosophers simultaneously pick up their left chopstick.

Deadlock Avoidance Strategies

- **Strategy 1:** Allow at most 4 philosophers to sit at the table simultaneously.
- **Strategy 2:** Allow a philosopher to pick up chopsticks only if both are available at once (checked inside a critical section — atomic pick-up of both).

- **Strategy 3 (Asymmetric Solution):** Odd-numbered philosophers pick up the left chopstick first, then the right. Even-numbered philosophers pick up the right first, then the left. This breaks circular wait.

Solution 2 — Monitor-Based Solution

A philosopher picks up chopsticks only if neither neighbour is eating, using the monitor `DiningPhilosophers` with `state[5]` and condition variables `self[5]` as described in Q4.

This solution guarantees:

- No two adjacent philosophers eat simultaneously (mutual exclusion).
- No deadlock — a philosopher is never blocked waiting for a chopstick held by someone also waiting.
- No starvation — every hungry philosopher eventually gets to eat.

QUICK REVISION — 2-MARK ANSWERS SUMMARY

Unit I — Operating System Overview & Structures

Q.No	Question	Short Answer
1	What is an OS?	System software acting as intermediary between user and hardware; manages resources.
2	Define Kernel.	Core part of OS, resident in memory, provides process/memory/device management services.
3	What is multiprogramming?	Multiple programs reside in memory; CPU switches among them to maximise utilisation.
4	How does dual-mode work?	Hardware provides user mode (bit=1) and kernel mode (bit=0); system calls switch modes.
5	Objectives of OS?	Convenience, efficiency, and ability to evolve.
6	Use of fork() and exec()?	fork() creates a child process; exec() replaces the process memory with a new program.
7	Define open-source OS.	OS whose source code is freely available for use, modification and redistribution (e.g., Linux).
8	List OS services.	UI, program execution, I/O ops, file-system, communication, error detection, resource allocation.
9	Define system calls.	Programming interface through which a user program requests a service from the OS kernel.
10	Purpose of system programs?	Provide a convenient environment for program development and execution (editors, compilers, loaders).

Unit II — Process, Threads & CPU Scheduling

Q.No	Question	Short Answer
1	Define process.	A program in execution; active entity with code, data, stack, heap and current activity.
2	State of a process?	Current activity — one of New, Ready, Running, Waiting, or Terminated.
3	Define scheduler.	OS module that selects processes from a queue and allocates CPU or memory to them.
4	Requirements for CS solution?	Mutual exclusion, progress, and bounded waiting.
5	Types of scheduler?	Long-term (job), short-term (CPU), and medium-term scheduler.
6	Define thread.	Basic unit of CPU utilisation (LWP); shares code/data with parent, has own PC, registers, stack.
7	What does PCB contain?	Process state, PID, PC, CPU registers, scheduling info, memory info, accounting, I/O status.
8	What is a multiprocessor?	System with 2+ CPUs sharing common memory and peripherals, working simultaneously.
9	Define context switching.	Saving state of current process in PCB and loading saved state of next process.
10	List thread libraries.	POSIX Pthreads, Win32 threads, Java threads.

Unit III — Process Synchronization & Deadlocks

Q.No	Question	Short Answer
1	Define deadlock.	Two or more processes blocked forever, each waiting for a resource held by another in the set.
2	Define race condition.	Outcome of concurrent shared data access depends on execution order — leads to inconsistent results.
3	Requirements of CS solution?	Mutual exclusion, progress, and bounded waiting.
4	Define semaphore.	Integer synchronisation variable accessed only through atomic wait() and signal() operations.
5	Classic sync problems?	Producer-Consumer (bounded buffer), Readers-Writers, Dining Philosophers.
6	What is critical section?	Code segment accessing shared resources; must not be executed by two processes simultaneously.
7	Busy waiting and spinlock?	Busy waiting = looping while checking a condition; a semaphore using busy waiting is a spinlock.
8	What is process sync?	Mechanisms to coordinate cooperating processes sharing data to preserve consistency and avoid races.

CPU Scheduling Formula Reference

Formula	Expression
Turnaround Time	Completion Time – Arrival Time
Waiting Time	Turnaround Time – Burst Time
Response Time	Time of first CPU response – Arrival Time
Average Waiting Time	Sum of all Waiting Times / Number of Processes
CPU Utilisation	Keep CPU as busy as possible (40%–90%)
Throughput	Number of processes completed per unit time
FCFS — Max Efficiency	When shortest process arrives first (no convoy effect)
SJF — Avg Wait	Minimum average waiting time among all non-preemptive algorithms
